Deep-Neural Genetic Algorithm Optimization Analysis for Forgery Detection of Banknotes

Sashank Sridhar
Department of Computer Science and Engineering
College of Engineering Guindy, Anna University
Chennai, India
sashank.ssridhar@gmail.com

Sowmya Sanagavarapu
Department of Computer Science and Engineering
College of Engineering Guindy, Anna University
Chennai, India
sowmya.ssanagavarapu@gmail.com

2021 13th International Conference on Electronics, Computers and Artificial Intelligence (ECAI) | 978-1-6654-2534-6/21/$31.00 ©2021 IEEE | DOI: 10.1109/ECAI52376.2021.9515040

***Abstract*— Metaheuristic algorithms aim to find high performing, near-optimal solutions with reasonable computing costs. Using the process of natural selection that belongs to a set of evolutionary algorithms, are Genetic Algorithms. These algorithms intelligently exploit random search over the data in search of a solution space for better performance. Detection of forgery in legal documents by an automated system remains a valid problem of today. In this paper, the identification and detection of forgery in the monetary notes is performed by using deep neural-based Genetic Algorithms. An Artificial Neural Network was used for training on the banknote dataset and the learned weights were vectorized for Genetic Algorithm processing. Genetic Algorithm is explained in detail along with the working of the Fitness model that determines the efficiency of the trained model. The model achieved a high-performance accuracy of 94% with the fitness score of 95.2. The improvement of the model with the infusion of trainable weights from a deep neural model is represented and analyzed.**

Keywords*— *Neural Optimization, Genetic Algorithm, Artificial Neural Network, Mutation, Crossover, Fitness Functions, Bank Note Authentication, Forgery Detection

I. INTRODUCTION

Counterfeit notes [1] refer to currency that is produced without the legal sanction of the government to mimic the original note for deceiving the recipient. The prevalence of counterfeit notes is harmful to the economy [2] of a nation as their circulation reduces the value of real money. Companies also end up losing their buying power due to lack of reimbursement of counterfeit notes. It is vital to have automated approaches [3] to identify defective notes as high quality counterfeits require manual examination for their identification, which is time-consuming and infeasible on a nation-wide scale.

There has been an extreme growth in the sheer volume of counterfeit money in recent years with the advancement of technology, especially with respect to production quality. Even with more innovative security features embedded into these notes, there have been many cases where the notes thwarted detection. In the UK 12 million EUR worth of counterfeit notes [4] was seized by the law enforcement agencies earlier this year. According to the India's National Crime Records Bureau (NCRB), fake currency totaling 287,404 notes worth 253 million INR was seized under Fake Indian Currency Notes (FICN) in 2019 — an increase of 11.7% over the previous year [5].

Machine Learning [6] helps in the design of systems for the detection and identification of counterfeit notes through feature extraction techniques. Supervised ML algorithms help optimize performance criteria using training to solve real-world computational problems [7]. Applications of these algorithms involved image and text processing for the detection of counterfeit notes by the analysis of features like watermarks, bleed lines, security thread and latent images [8].

Genetic Algorithm (GA) [9] is a stochastic search algorithm used in machine learning that creates multiple solutions to a given problem that evolve through a number of generations. Each of the derived solutions from the algorithm hold the trained weights that help optimize the output of the model. Their adaptive heuristic approach follows an iterative pattern that changes with time to simulate behaviors of entities in supervised methods. The learning process of this set of evolutionary algorithms [10] is accelerated by coupling deep neural networks with them. The trained weights of the neural network [11] in the hidden layers after training are converted from weighted matrices to single dimensions vectors. These vectors are concatenated together with selection and pooling operations for their high performance.

In this paper, Genetic Algorithm modelling has been infused with the trained weights from Artificial Neural Networks for the identification of the counterfeit notes in the banknote dataset. By combining the performance of the deep neural model with the working of Genetic algorithm involving selection of features through a fitness module for pooling. Fitness calculator of a point aids the algorithm in to move between points in search space for a possibly better solution. After hyper tuning of the parameters of the implemented model, a high classification accuracy was achieved and results were analyzed.

The rest of the paper is organized as follows. Section II gives the summary of some of the best works in banknote counterfeit detection using machine learning as well as neural modelling. The design of the system using Genetic Algorithm to optimize neural networks is given in Section III. The implementation details of the constructed neural model with genetic algorithm optimization is given in Section IV. The results obtained from the forgery detection model along with the analysis are present in Section V. The summarization of the project and the proposed future work is given in Section VI.

II. RELATED WORKS

Some of the best works for banknote authentication using machine learning and deep learning based models are discussed here.

Genetic algorithm supported neural network systems have been implemented on banknote authentication dataset to find the optimal set of weight vectors from its training. In this machine learning technique by Sarma [12], the initial weight vector to the input layer of the constructed neural model has been optimized gradually for enhancing its performance. Features from the dataset were extracted using Bayes classifier for minimum error in the class descriptions. The results obtained from the system were compared with the individual results of the multi-layer perceptron with feed-forward network and a simple neural model.

978-1-6654-2534-6/21/$31.00 ©2021 IEEE

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 07:00:41 UTC from IEEE Xplore. Restrictions apply.

Genetic Algorithm based polynomial neural network (PNN-GA) modelling was implemented by Nayak et al [13]. The architecture of the PNN was similar to a Feed-Forward Neural network with the replacement of the neurons with Polynomial nodes, where the output of each node is a higher order polynomial. The performance of the model was analyzed by varying the number of input variables from the dataset and the order of polynomials which are determined by hyperparameter tuning. Comparison of the PNN-GA was done with Back Propagation based PNN (PNN-BP) and the results were presented.

Automated system for the detection of fake currency notes using supervised machine learning algorithms was implemented by Yadav et al [14]. They performed a quantitative analysis of the algorithms on the basis of various evaluation parameters for measuring their performance. By studying the correlation between the features and target label of the dataset, high performance results were obtained. Multiple train-test split ratios were used in the testing phase and the results were analyzed.

Image processing techniques have also been used in machine learning algorithms to identify counterfeit notes. Bhatia et al [15] used a fake currency classifying dataset developed using computational and mathematical entities like color, shape, paper width. The k-Nearest Neighbors algorithm which was implemented for classifying the data in a suited category using Euclidean distance to measure similarity, outperformed the support vector classifier and gradient boosting algorithm. k-Fold cross validation was finally performed for the analysis of the model's performance.

Systems for classification of counterfeit notes also used certain security features like embedded latent image and security thread. Singh et al [16] proposed a unique authentication system to extract and encode these features from Indian banknotes in image planes using a clustering algorithm. Histogram of Oriented Gradients (HOG) descriptor, primarily used in computer vision for identifying edge directions, was used for encrypting the security features using template matching and classified using the Support Vector Machines classifier.

In summary, our model dynamically updates weights of the neural network by applying genetic algorithm to find weights with the best possible accuracy. Optimization is done by applying a fitness function on the population to filter the best possible mating pool. Offspring are generated with features of both parents from the mating pool by applying crossover. To improve the feature set of the offspring, mutations are applied. Over the course of generations, the most optimal solution is obtained. By using a deep neural network in place of a probabilistic model like Bayes, feature matrices are created which try to model the inputs accurately. In the neural network, we have implemented mutation and crossover functions on the feature set to optimize the weights in the matrices from the input data. By using a semi-supervised learning model, the initial matrices are assigned randomly and they are optimized through training based on the fitness function. This provides an advantage over fully supervised models in creating a pool of population to determine the best possible outcome from them in place of using a single point of learning. The dataset comprises features obtained from wavelet transformation of original images. These features are the discrete characteristics derived from the continuous range of inputs. In our application, wavelet transformation of any currency note can be applied to determine the originality of the note.

III. System Design

The overall design of the GA optimized neural networks is given in Figure 1. The weights of the neural network are trained based on the variations introduced in the GA and a fitness function evaluates the feasibility of the mutated solution at each round of generation. The best solution achieved in terms of accuracy will be assigned as the weights of the neural network.

A. Dataset Description

The dataset used in this paper is collected from [17] in the UCI Machine Learning Repository. It comprises features extracted from images of both genuine and forged banknote specimens. Wavelet transformation [18] is applied on the images and the features from Table I are calculated. Each image is assigned a class 0 if it is real and a class 1 if it is counterfeit.

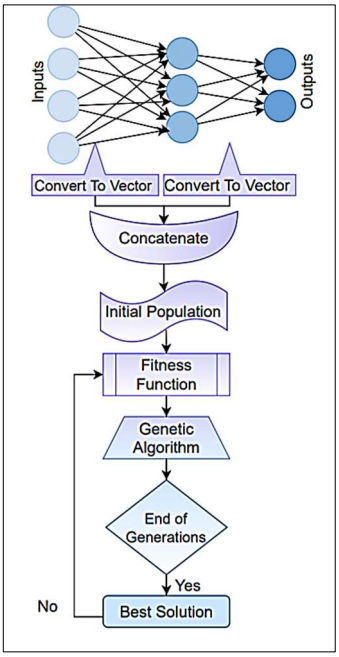


Fig. 1. Overall System Design of the GA Optimized Neural Network

TABLE I. Features Extracted from Wavelet Transformed Images

Feature Name	*Description*
Variance	Spread of the distribution values about the mean
Skewness	Asymmetrical deviations from the symmetric bell curve of the distribution
Kurtosis	Relative weight of the tail of the distribution compared to the center
Entropy	Relative weight of the tail of the distribution compared to the center

B. Creation of Initial Population

The first step involved in optimizing the weights of a neural network through GA is to set up the initial population. The number of parameters P_l at a single step of generation t depends on the number of solutions k created at that step and is calculated using equation (1) for a fully connected feed forward neural network. Solutions refer to different possible matrices of weights in the network.

$$P_t = k * \sum_{i=1}^{m} n_{i-1} * n_i \quad (1)$$

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 07:00:41 UTC from IEEE Xplore. Restrictions apply.

Where m is the total number of layers in the neural network and $n_{0\ldots(m-1)}$ is the number of nodes in each of the m layers.

Each layer l in the neural network has a corresponding weight matrix W_l and the number of parameters in that matrix is given by equation (2).

$$W_l = n_{l-1} * n_l \quad (2)$$

Each of these weight matrices are converted to one dimensional vector and serve as chromosomes for the genetic algorithm. The chromosomes in a GA refer to the feature sets of the individual solutions. The vector representation of the weight matrices is concatenated together and serve as a single one-dimensional solution.

C. Genetic Algorithm

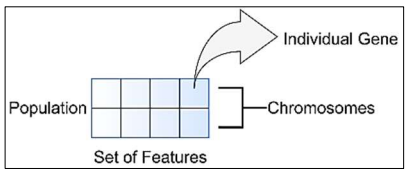


Fig. 2. Structure of Chromosomes and Genes

Genetic algorithms involve chromosomes and genes as seen in Figure 2. Chromosomes correspond to each set of features and genes correspond to a particular individual feature [19]. A set of chromosomes form a population.

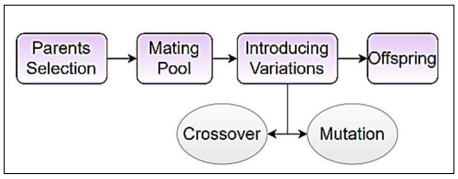


Fig. 3. Working of Genetic Algorithm

The first step in GA is to select the parents is in Figure 3. Parents are individuals whose features are combined to produce offspring. The selected parents form a mating pool where every two parents generate two offspring. By using a fitness function, the parents are evaluated and only those parents with a high fitness value will be retained in the mating pool. By iteratively applying a fitness function, only the best solutions are retained. In order to ensure that the offspring do not retain negative attributes of parents, variations are introduced. They can be of the form crossover or mutations. Crossovers introduce offspring with characteristics of both the parents whereas mutations will change the values of the features to improve the fitness score.

D. Fitness Function

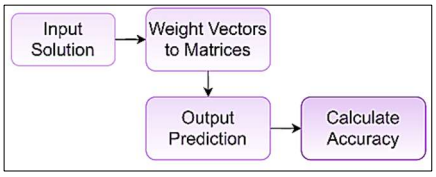


Fig. 4. Calculating Fitness of a Solution using the Fitness Function

Each solution at every iteration is evaluated using a fitness function [20] given in Figure 4. Based on the fitness value generated the solutions are ranked and the mating pool is determined. Fitness function used in a classification task is accuracy. The inputs are passed through the weight matrices and accuracy is calculated based on the output predictions. The solutions are sorted for the selection of parents to the mating pool.

In this way the weight matrices are optimized either using crossover or mutation on the parent matrices. The optimized weight matrices are evaluated at each step and a fitness score is calculated to determine the best possible updates for the matrices.

IV. System Implementation

The implementation details of the system are given in this section. The GA optimized neural network is set up on Google Colab with Intel(R) Xeon(R) CPU @ 2.20GHz Processor, 13 GB RAM and 12GB NVIDIA Tesla K80 graphics processor.

A. Dataset Processing

The data collected is first normalized [21] to ensure that all the input values are in the range [0,1]. This is to ensure that the convergence of the model to the local minima takes place faster during training. Min-Max normalization is carried out using equation (3).

$$\bar{X} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (3)$$

The data set is then split to a train-test set with the ratio 80:20. Table II shows the number of instances present in the training and testing set.

TABLE II. Dataset Distribution

	Real instances	***Counterfeit instances***
Training set	604	493
Testing set	158	117

B. Neural Network Structure

The neural network structure developed has an input layer with 150 nodes, one hidden layer with 60 nodes and an output layer with 2 nodes each corresponding to the class 0 or 1 depending on whether the bank notes are real or counterfeit. There are 3 weight matrices present in the network : between the inputs and the input layer of the size $Number\ of\ input\ instances * 150$, between the input layer and the hidden layer of the size $150 * 60$ and between the hidden layer and the output layer of the size $60 * 2$. The matrices are initially assigned random values in the range [-4.0, 4.0]. The number of nodes in the neural network is higher than the available features in the dataset for a bigger selection pool of chromosomes to improve the chances of better crossover and mutation.

C. Creating Initial Population from Weight Matrices

The initial population is generated by vectorizing the weight matrices assigned to the neural network. The number of solutions in the population is taken as 20. For each solution, a set of 3 weight matrices is assigned. The population vector is calculated using Algorithm 1. The initial loop iterates through each solution and the inner loop converts each weight matrix in that solution to a vector. The vectors for a particular solution are all concatenated together to form a 1D representation of the solution. All solutions together form the initial population vector.

D. Calculating Fitness

After the population is defined, fitness scores are calculated on the population in order to determine the best possible mating pool of parents. The fitness function used is

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 07:00:41 UTC from IEEE Xplore. Restrictions apply.

accuracy. The accuracy of classification is determined by making predictions on the input data set as seen in Algorithm 2. The fitness function as shown in equation (4) is applied on the predicted classes.

$$Fitness = Correctly\ Predicted\ Classes\ /\ Total\ Number\ of\ Inputs \quad (4)$$

Algorithm 1 : Creating Initial Population

```
input : n_s: Number of Solutions, n_weights: Number of weight
        matrices, W_1..n_weights: Weight Matrices
output: population_vector : Output Population Vector
Function Create Initial Population(n_s, n_weights,
 W_1..n_weights):
    population_vector ← []
    for i in range(n_s) do
        vector ← []
        for j in range (n_weights) do
            vector.extend(array(W_j))
        end
        population_vector.append(vector)
    end
    population_vector ← array(population_vector)
end
```

Algorithm 2. Fitness Score Calculation

```
input : n_weights: Number of weight matrices, W_1..n_weights:
        Weight Matrices, data: Input data, data_class: Expected
        class, activation: Activation Function
output: prediction_class : Predicted Labels
Function Fitness Score Calculation(n_weights, W_1..n_weights,
 data, data_class, activation):
    prediction_class ← []
    for input in data do
        for i in range (n_weights) do
            forward_activation ←input*W_i
            forward_activation ←activation(forward_activation)
        end
        forward_activation ←sigmoid(forward_activation)
        prediction_class.append(indexof(max(forward_activation)))
    end
end
```

E. Selection of Mating Pool

For each solution in the population vector, fitness score is evaluated and a mating pool is selected based on best fitness score as seen in Algorithm 3.

Algorithm 3. Select Mating Pool

```
input : population_vector: Population, n_parents: Number of
        Parents
output: mating_pool : Selected Parents
Function Mating Pool(population_vector, n_parents):
    mating_pool ← []
    fitness ← []
    for individual in population do
        fitness.append(Fitness_Score_Calculation(individual)
    end
    for i in range (n_parents) do
        max_fitness ←indexof(max(fitness))
        mating_pool.append(population_vector[max_fitness])
        fitness.remove(max_fitness)
    end
end
```

F. Crossover

Offspring are generated from the parents in the mating pool by performing crossover between the parents. Algorithm 4 shows how crossover is applied. A crossover point is determined which is half the length of the number of features (genes) in the chromosomes. Each offspring will have some features from both parents. The mating pool size in the implementation was chosen as 10. The parents of the kth offspring were $k\%10$ and $(k+1)\%10$ individuals in the mating pool.

G. Mutation

After the offspring are generated, the features of each offspring are varied slightly to get a new set of features. In this implementation, the percentage of mutation is determined to be 10%. As seen in Algorithm 5, the number of mutations is calculated as a percentage of the number of features and the mutation indices are determined randomly. At each mutation index, the existing feature is replaced with a random value within a range of (-1.0, +1.0). In this way, new features are added to the offspring.

Algorithm 4: Crossover Variation

```
input : mating_pool : Selected Parents, n_parents: Number of
        Parents, n_offspring: Number of Offspring, n_features:
        Number of Features
output: offspring: Offspring Vectors
Function Crossover(mating_pool,n_parents,n_offspring,
 n_features):
    offspring ← []
    crossover ←n_features/2
    for k in n_offspring do
        parent1_idx ←k%n_parents
        parent2_idx ←(k+1)%n_parents
        offspring[k, 0 : crossover] ←mating_pool[parent1_idx,
         0:crossover]
        offspring[k, crossover :] ←mating_pool[parent2_idx,
         crossover:]
    end
end
```

Algorithm 5: Mutation of Offspring

```
input : offspring: Offspring Vectors, mutation_percent:
        Percentage of mutation, n_features: Number of Features
output: mutation_offspring: Mutated Offspring
Function Mutation(offspring,mutation_percent,n_features):
    num_mutations ←(mutation_percent * n_features)/100
    mutation_indices ←random(range(0, n_features),
     num_mutations)
    mutation_offspring ← []
    for idx in n_offspring do
        random_value ←random.uniform(-1.0, 1.0, 1)
        mutation_offspring[idx, mutation_indices] =
         mutation_offspring[idx, mutation_indices] +
         random_value
    end
end
```

H. Overall Flow

GA is applied iteratively with the steps of selecting the mutation pool, generating offspring from the pool through crossover and then applying mutations on the generated offspring. At the beginning of each step of generation, fitness scores are calculated by applying accuracy and the solution with the best score is determined at the end of the generation cycle. Parameters used for the GA are summarized in Table III.

TABLE III. SUMMARY OF PARAMETERS USED

Parameter	*Value*
Number of Solutions	20
Number of Parents	10
Number of Offspring	10
Percentage of Mutation	10
Number of Generations	100
Activation Function	ReLu
Avg. Time taken for each generation	1.2 sec

V. RESULTS AND ANALYSIS

The results obtained from the implemented neural network based Genetic Algorithm (GA) framework is presented below along with their analysis.

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 07:00:41 UTC from IEEE Xplore. Restrictions apply.

A. *Variation of Fitness of GA with Number of Generations*

The calculation of the fitness score in the implemented deep-neural GA model is given here. The number of weight matrices in this module will be determined based on the number of layers and its size is based on the number of nodes. The weight matrices for each layer are concatenated together to form a solution. The number of solutions is chosen based on the performance of the model after hyperparameter tuning as 20. The matrix for a layer has dimensions $(n-1)*n$ where $(n-1)$ is the number of nodes present in the previous layer and n is the number of nodes in the current layer. Initially, random values between a chosen range are applied to each element in the matrix, here [-4.0,+4.0] is chosen. The matrix multiplication is performed and the output weight is taken as the prediction value for the class using a sigmoid function. The accuracy of the predictions are recorded and the highest score of accuracy is used as the fitness score for the model at the nth iteration. The variation of the fitness score with the increasing number of iterations is recorded in Figure 5. The training was done for 100 iterations and the fitness score of the system reached 95.168 at the final iteration.

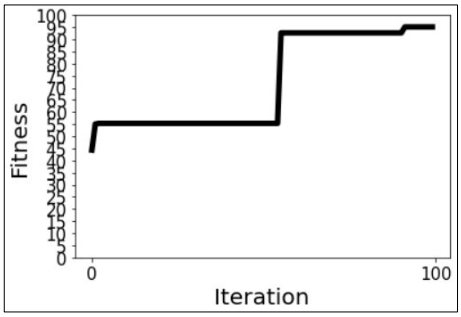


Fig. 5. Variation of Fitness Score with Number of Iterations

B. *Confusion Matrix obtained from the neural-based GA model*

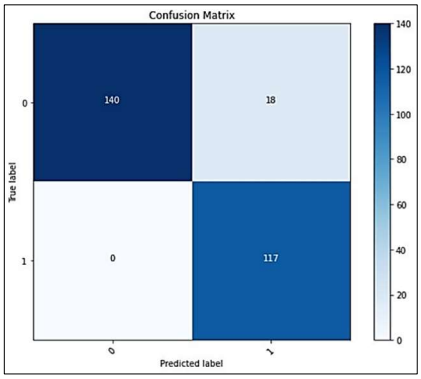


Fig. 6. Test Set Confusion Matrix of the GA Optimized Neural Model

The confusion matrix gives the distribution of the prediction outputs over the class label along with their distribution. The class labels indicated whether the banknotes were counterfeit or real. Class-1 or Positive class indicated the banknote was counterfeit and Class-0 or Negative class indicated the banknote was non-counterfeit i.e., real. In Figure 6, the confusion matrix of the implemented deep-neural based GA algorithm for detection of counterfeit notes is given. It is observed that the implemented model has high performance accuracy with only few of the actual notes marked as forged (false negative).

C. *Performance measure of the GA model with the Banknote Authentication dataset*

The detection of the counterfeit notes was carried out by the ANN-based Genetic Algorithm model that was evaluated using Accuracy, Precision, Recall and F1-score. The testing values obtained from the implemented model is given in Figure 7. The testing accuracy of the model reached 94% indicating the high performance of the model.

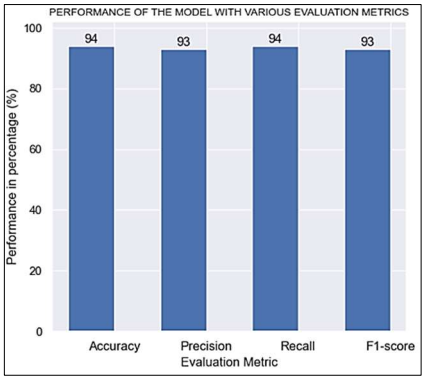


Fig. 7. Performance in percentage of the model with evaluation metrics

Accuracy [22] gives the number of times the constructed GA model was able to predict the positive or negative class correctly out of the total number of predictions, in eq.(5).

$$Accuracy = (TP + TN)/(TP + TN + FP + FN) \quad (5)$$

Precision [22] gives the measure of the proportion of positive identifications by the model that were actually correct, calculated using eq. (6).

$$Precision = TP/TP + FP \quad (6)$$

The value of recall [22] gives what proportion of the actual positive value were identified correctly, calculated using eq. (7).

$$Recall = TP/TP + FN \quad (7)$$

For binary classification, F1-Score [22] or balanced F-score or F-measure refers to the test's accuracy. It is derived here from the weighted average of Precision and Recall as given in eq. (8).

$$F1\ Score = 2 * Precion * Recall/(Precision + Recall) \quad (8)$$

D. *ROC and AUC for the Deep-neural GA model*

The Region of Convergence (ROC) [23] curve plots parametrically $TPR(T)$ versus $FPR(T)$ with T as the varying parameter. The actual shape of the ROC curve is determined by how much overlap the two distributions have. The ROC Curve for the trained deep-neural GA model is plotted in Figure 8. The Area Under the Curve (AUC) [23] is equal to the probability that a classifier will rank a randomly chosen positive instance higher than a randomly chosen negative one, if 'positive' is ranked higher than 'negative' and vice versa. It is calculated that the area under the curve is 0.943, indicating the high performance of the model in the testing.

E. *Comparison analysis of the perfromance of the Deep-Neural GA model*

The performance of the implemented Deep-neural based GA model for banknote authentication is compared with the existing state-of-the-art models, in Table IV. It is observed

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 07:00:41 UTC from IEEE Xplore. Restrictions apply.

that the proposed model outperforms the existing model with an accuracy of 94%.

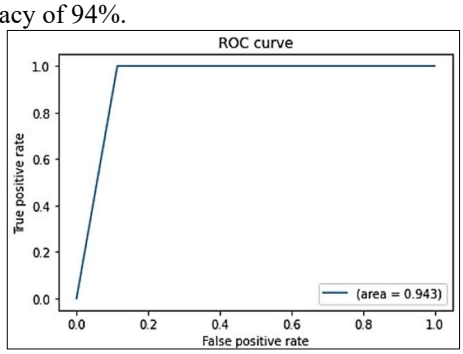


Fig. 8. ROC Curve with the AUC value of the GA Optimized Neural Model

TABLE IV. COMPARISON OF THE DEEP-NEURAL GA MODEL WITH EXISTING BANK AUTHENTICATION MODELS

Implementation	Accuracy
RBF by Kumar and Dudyala [24]	57.0388%
PNN-BP by Nayak et al. [13]	79.0%
Naive Bayes by Yadav et al. [14]	83.3%
Naïve Bayes by Kumar and Dudyala [24]	85.9%
PNN-GA by Nayak et al. [13]	87.0%
Our Implementation (Deep-neural GA)	**94%**

VI. CONCLUSION AND FUTURE WORKS

Genetic Algorithm is a Machine Learning algorithm that generates a number of solutions over generations and determines to arrive at a solution that gives the maximum fitness value. The calculation of the fitness module is done to determine the best possible mating pool of parents using the trained matrix weights, vectorized from the learned weights from the training of the Deep Neural network. A dataset containing the detection of counterfeit notes was used to study the working of the implement deep neural Genetic Algorithm and the evaluation of the model is done over a number of performance metrics. The deep neural Genetic Algorithm guarantees highly optimized solutions for any complex dataset or datasets with less trainable features, like in this case. The feature selection performed through selection of solutions could be optimized to explore the applications of the model in hyperparameter tuning of trained deep neural networks to build robust systems.

REFERENCES

[1] K. Kamble, A. Bhansali, P. Satalgaonkar, and S. Alagundgi, "Counterfeit Currency Detection using Deep Convolutional Neural Network," in 2019 IEEE Pune Section International Conference (PuneCon), pp. 1–4, 2019.

[2] V. Butticè, F. Caviggioli, C. Franzoni, G. Scellato, P. Stryszowski, and N. Thumm, "Counterfeiting in digital technologies: An empirical analysis of the economic performance and innovative activities of affected companies," Research Policy, vol. 49, no. 5, 2020.

[3] S. Arya and Dr. M. Sasikumar, "Fake Currency Detection," in 2019 International Conference on Recent Advances in Energy-efficient Computing and Communication (ICRAECC), pp. 1–4, 2019.

[4] W. Taylor, "'Largest ever haul of fake money' seized as three jailed over £12 million counterfeit scheme," Yahoo Finance, Jan. 2021. https://in.finance.yahoo.com/news/counterfeit-money-seized-police-120605811.html (accessed Jun. 02, 2021).

[5] P. Gill, "The number of fake ₹500 notes in India increased by 31% in the last one year," Business Insider, May 2021. https://www.businessinsider.in/finance/banks/news/the-number-of-fake-500-notes-in-india-increased-by-31-in-the-last-one-year/articleshow/83030750.cms (accessed Jun. 02, 2021).

[6] M. Jadhav, Y. K. Sharma, and G. M. Bhandari, "Currency Identification and Forged Banknote Detection using Deep Learning," in 2019 International Conference on Innovative Trends and Advances in Engineering and Technology (ICITAET), pp. 178–183, 2019.

[7] P. Dhapare, A. Agarwal, and D. Doshi, "Detection of Counterfeit Currency using Image Processing Techniques," in 2019 IEEE 5th International Conference for Convergence in Technology (I2CT), pp. 1–5, 2019.

[8] S. V. Viraktamath, K. Tallur, R. Bhadavankar, and Vidya, "Review on Detection of Fake Currency using Image processing Techniques," in 2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS), pp. 865–870, 2021.

[9] N. Paenoi and S. Sitjongsataporn, "Automatic Transfer Function Improvement based on Genetic Algorithm," in 2021 7th International Conference on Engineering, Applied Sciences and Technology (ICEAST), pp. 230–233, 2021.

[10] I. S. Donbosco and U. K. Chakraborty, "Comparing Performance of Evolutionary Algorithms - A Travelling Salesman Perspective," in 2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence), pp. 182–187, 2021.

[11] O. Üstün, E. Bekiroğlu, and M. Önder, "Design of highly effective multilayer feedforward neural network by using genetic algorithm," Expert Systems, vol. 37, no. 4, 2020.

[12] S. S. Sarma, "Bank Note Authentication: A Genetic Algorithm Supported Neural based Approach," International Journal of Advanced Research in Computer Science, vol. 7, no. 7, pp. 97–102, 2016.

[13] J. Nayak, N. Sahoo, J. R. Swain, T. Dash, and H. S. Behera, "GA Based Polynomial Neural Network for Data Classification," in 2014 International Conference on Information Technology, pp. 234–239, 2014.

[14] A. Yadav, T. Jain, V. K. Verma, and V. Pal, "Evaluation of Machine Learning Algorithms for the Detection of Fake Bank Currency," in 2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence), pp. 810–815, 2021.

[15] A. Bhatia, V. Kedia, A. Shroff, M. Kumar, B. K. Shah, and Aryan, "Fake Currency Detection with Machine Learning Algorithm and Image Processing," in 2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS), pp. 755–760, 2021.

[16] M. Singh, P. Ozarde, and K. Abhiram, "Image Processing Based Detection of Counterfeit Indian Bank Notes," in 2018 9th International Conference on Computing, Communication and Networking Technologies (ICCCNT), pp. 1–5, 2018.

[17] V. Lohweg and H. Doerksen, "Banknote Authentication Data Set," UCI Machine Learning Repository, 2013. https://archive.ics.uci.edu/ml/datasets/banknote+authentication (accessed Jun. 03, 2021).

[18] A. Chhabra, D. Kim, and K. Cohen, "Enhanced Cascaded Genetic Fuzzy System for Counterfeit Banknote Detection," in North American Fuzzy Information processing Society NAFIPS'2020, pp. 1–11, 2020.

[19] S. Katoch, S. S. Chauhan, and V. Kumar, "A review on genetic algorithm: past, present, and future," Multimedia Tools and Applications, vol. 80, pp. 8091–8126, 2021.

[20] J.-H. Han, D.-J. Choi, S.-U. Park, and S.-K. Hong, "Hyperparameter Optimization for Multi-Layer Data Input Using Genetic Algorithm," in 2020 IEEE 7th International Conference on Industrial Engineering and Applications (ICIEA), pp. 701–704, 2020.

[21] Q.-F. Wang, K. Yao, R. Zhang, A. Hussain, and K. Huang, "Improving deep neural network performance by integrating kernelized Min-Max objective," Neurocomputing, vol. 408, pp. 82–90, 2020.

[22] S. Sridhar and S. Sanagavarapu, "Fake News Detection and Analysis using Multitask Learning with BiLSTM CapsNet model," in 2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence), pp. 905–911, 2021.

[23] S. Sanagavarapu and S. Sridhar, "Network Routing in SDNs Using Topology Based Multitask Neural Modelling," in 2021 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS), pp. 1–9, 2021.

[24] C. Kumar and A. K. Dudyala, "Bank note authentication using decision tree rules and machine learning techniques," in 2015 International Conference on Advances in Computer Engineering and Applications, pp. 310–314, 2015.

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 07:00:41 UTC from IEEE Xplore. Restrictions apply.